

Mirror: A Neuro-Symbolic Architecture for Persistent Epistemic Integrity in LLM Interactions

Stanley Mao
Independent Researcher
Sydney, Australia
stanmao@gmail.com

April 2026

Abstract

Large language models exhibit well-documented failure modes over extended interactions: semantic drift, plausibility-filling, and unconstrained escalation toward operational action. Existing approaches address these individually—retrieval-augmented generation for memory, constitutional AI for value alignment, chain-of-thought prompting for reasoning quality—but no unified architecture addresses all three simultaneously in a model-agnostic framework. This paper presents Mirror, a neuro-symbolic architecture providing constraint mechanisms for epistemic integrity, persistent semantic coherence, and agent safety across long-horizon LLM interactions. Mirror comprises three integrated subsystems: (1) OLI (Operational Layer Integrity), a constitutional constraint system with hard-gated claim admissibility; (2) a graph-of-graphs semantic memory corpus enabling persistent context; and (3) a layer integrity mechanism bounding agent action scope. The architecture operates as a prompt-layer scaffolding with code-side post-generation validation, applicable to any frontier LLM without weight modification. We describe the architecture, a working implementation (Prime’s Shadow), preliminary operational data from 14 sessions and 409 logged pipeline events, known limitations, and directions for empirical evaluation.

1 Introduction

The deployment of large language models in extended, high-stakes reasoning tasks has surfaced a class of failure modes that prompt engineering alone cannot reliably prevent. Three are particularly consequential:

Semantic drift: the gradual degradation of conceptual precision over long interaction sequences, as the model’s effective context window fills and earlier constraints lose influence. This is distinct from factual error; the model may remain factually accurate while progressively abandoning the epistemic posture or constraint framework established at session initialisation.

Plausibility-filling: the tendency of LLMs to accept or generate reasoning that is internally coherent but epistemically unsupported—filling evidential gaps with confident-sounding inference. This is particularly hazardous in research and decision-support contexts, where the appearance of epistemic rigour may be indistinguishable from its substance to a non-expert reader.

Operationalisation drift: the incremental escalation of model outputs from analysis toward tactical coordination and action-planning, particularly dangerous in agentic deployment contexts. This can emerge from cumulative problem-solving pressure, user frustration, or narrative momentum without explicit instruction.

Current mitigations are fragmented. RAG systems [Lewis et al., 2020] address retrieval but not constraint. Constitutional AI [Bai et al., 2022] addresses values but not epistemics. Memory

systems address persistence but not semantic integrity. No existing framework provides a unified, deployable solution to all three simultaneously.

Mirror emerged from sustained applied research into these failure modes through iterative interaction with frontier LLMs. What began as a prompt engineering exercise evolved into a layered architectural specification as each mitigation revealed the need for deeper structural intervention. The result is a model-agnostic scaffolding system that treats the LLM as a neural substrate to be constrained by symbolic rules—a neuro-symbolic hybrid in the classical sense [Marcus, 2020, Garcez and Lamb, 2019].

This paper makes four contributions:

1. A formal specification of OLI (Operational Layer Integrity), a constitutional constraint layer with claim admissibility gating and resistance-detection invariants.
2. A graph-of-graphs semantic memory architecture (the Corpus) enabling persistent, version-controlled context across sessions.
3. A layer integrity mechanism (L0–L4) providing bounds on agent action scope with slope-detection and escalating intervention.
4. A working open-source implementation (Prime’s Shadow) integrating all three subsystems with a Python backend, code-side post-generation validator, and web-based interface.

2 Background and Motivation

2.1 The Semantic Drift Problem

Standard transformer-based LLMs process context as a flat token sequence. Over extended interactions, this produces measurable degradation in the model’s effective adherence to early session constraints—a phenomenon we term *semantic drift*. Existing work on long-context models (extended context windows, sliding window attention) addresses the mechanical capacity for context but not the semantic entropy that accumulates within it. A 1M-token context window does not prevent a model from effectively downweighting a constraint established at token 1 by the time it reaches token 800,000.

2.2 Plausibility-Filling as an Epistemic Hazard

LLMs are trained to produce fluent, coherent continuations. This optimisation objective creates a systematic bias toward plausible-sounding outputs even in the absence of sufficient evidential grounding. We observe this when models reason about uncertain or under-constrained domains: rather than acknowledging epistemic limits, they generate confident-sounding inferences that conflate INFERENCE with FACT.

A particularly insidious form is *self-sealing logic*—reasoning structured to absorb rather than acknowledge counterevidence. Self-sealing outputs are epistemically indistinguishable from rigorous reasoning to a non-expert reader, making them especially hazardous in decision-support contexts. OLI’s epistemic floor (Section 3.1) explicitly prohibits self-sealing construction: any argument that cannot in principle be falsified is inadmissible regardless of its internal coherence.

2.3 Operationalisation Drift in Agentic Systems

As LLMs are deployed in agentic contexts—systems that take actions, use tools, or coordinate multi-step tasks—a new failure mode emerges: the gradual escalation of outputs from analysis (*describing what is*) through evaluation (*assessing what is good*) toward operationalisation (*planning how to act*). We term this *L4 drift*, after the layer taxonomy introduced in Section 3.5. L4 drift is not necessarily the result of explicit instruction; it can emerge from the cumulative

pressure of a problem-solving context, what the Mirror architecture terms *cinematic compression*—narrative momentum that compresses complex situations into action-ready framings—and *inevitability arcs*—reasoning trajectories that present a particular course of action as the only viable option.

2.4 System Lineage

Mirror, Prime’s Shadow, and Prime represent three maturity levels of the same architectural lineage. **Mirror** is the core architecture: a model-agnostic specification for constraint-aware reasoning, deployable immediately as a prompt-layer protocol on any frontier LLM. **Prime’s Shadow** is the working implementation described in this paper: a self-hosted Python application with a local LLM runtime, code-side OLI enforcement, YAML-backed corpus persistence, and a web-based interface with live corpus visualisation. **Prime** is the long-horizon target, extending the architecture with an external real-world model and full graph-database substrate—identified as future work. This paper focuses on Mirror (the architecture) and Prime’s Shadow (the implementation).

3 The Mirror Architecture

Mirror is structured as three integrated subsystems operating over a shared interaction stream. The overall pipeline processes each turn as follows:

Input → [Function Gate] → [Anchor Resolution] → [FrameState Load] → [OLI
Constraint Layer] → [Layer Integrity Check] → Output

Each incoming message is classified by function before any context loading or constraint checking occurs.

A note on “enforcement.” Because Mirror operates as a prompt-layer protocol with code-side validation rather than a weight-level modification, its constraint mechanisms depend in part on the model’s instruction-following fidelity. We use the term “enforcement” throughout to mean *specification of constraints with code-side detection and intervention*—not guaranteed prevention. The code-side validator provides a second line of defence that operates independently of the model’s compliance, but a model generating constraint-violating content in ways not covered by the validator’s heuristics could evade detection (see Section 6).

3.1 OLI: Operational Layer Integrity

OLI is a ten-layer constitutional constraint system (v2.1) applied to all non-trivial model outputs. The layers are ordered by foundational priority: each layer’s constraints are preconditions for the layers above it. The system is fail-closed—if a constraint cannot be satisfied, output is flagged or blocked rather than proceeding with degraded rigour.

Table 1 presents the full OLI specification. Several design properties warrant explicit discussion.

The **L0.5 hard gate** is the most operationally consequential layer. By requiring every non-trivial assertion to be explicitly typed before generation, it forces the model to surface its own epistemic status rather than allowing confidence to be implied by fluency. The UNKNOWN category is particularly important: it treats “I do not know” as a valid and complete output, resisting the helpfulness bias that causes LLMs to generate plausible-sounding content absent genuine knowledge. Each FACT claim requires an explicit *verification path*: a reproducible route by which the claim could be validated (e.g., citation, direct observation, supplied evidence, formal

derivation, dataset, or explicit calculation). Claims lacking a verification path must downgrade to INFERENCE or UNKNOWN—plausibility is not an admissible substitute for knowledge.

L1’s domain separation addresses a subtler failure mode: the contamination of external technical reasoning by internal affective or motivated states. By maintaining a hard boundary between the internal domain (which is logged as an ephemeral, turn-local buffer but not interpreted by default) and the external domain (which is subject to adversarial scrutiny), OLI prevents motivated reasoning from masquerading as technical analysis.

L6’s traceability requirements extend L0.5 by targeting specific linguistic patterns that signal vague authority—phrases like “research suggests” or “it is generally accepted” that imply evidential grounding without providing it. These are prohibited; every claim must trace to a specific, nameable source or downgrade to INFERENCE or UNKNOWN.

L8’s runtime refinement constraint addresses a subtle attack surface: the possibility that a user correction could retroactively legitimise prior reasoning. By scoping corrections to future turns only, OLI ensures that the epistemic status of past outputs is immutable—analogous to append-only audit logs in secure systems.

A **Final Override**—correctness over usefulness, unconditionally—sits outside the layer hierarchy. It represents the system’s acknowledgement that the value of an epistemic integrity framework comes precisely from its unconditional nature: a system that abandons correctness under sufficient pressure provides no meaningful safety guarantee.

OLI ON vs. OLI OFF. The system supports two operating modes. In **OLI ON** mode, all ten layers are active with mandatory claim tagging (L0.5) and full traceability (L6). In **OLI OFF** mode, a lightweight anti-confabulation posture is maintained: no mandatory tagging, but fabricated sources, vague authority, and gap-filling without evidential signal remain prohibited. OLI OFF provides a less friction-intensive interaction mode that preserves core epistemic hygiene.

3.2 Behavioural Mode Distribution

A constraint system that is technically rigorous but cognitively exhausting to interact with will not be used consistently—and inconsistent use of a safety mechanism is functionally equivalent to no mechanism at all. Mirror addresses this through a Behavioural Mode Distribution (BMD): a weighted allocation across reasoning archetypes that shapes the texture and style of responses without modifying model weights or relaxing epistemic constraints.

The BMD is defined as a probability distribution W over a set of reasoning archetypes $A = \{a_1, a_2, \dots, a_n\}$, where each archetype a_i is characterised by a primary reasoning axis and a weight w_i such that $\sum w_i = 1$. Routing logic adjusts weights dynamically based on interaction context, with the surface voice remaining unified.

Five reasoning axes are defined in Mirror v1.2 (Table 2). The BMD is explicitly *not* a claim about the model’s internal architecture or the existence of discrete reasoning modules—it is a practical mechanism for tuning communication style at the prompt layer.

Routing logic adjusts weights contextually: high abstraction elevates Structural Integrity and Scientific Rigour; novel or implausible hypotheses elevate Scientific Rigour and Variance Injection; detected rigidity elevates Cognitive Mobility; social or incentive-heavy content elevates Social Realism; escalation energy elevates Structural Integrity with a brief Variance Injection spike to interrupt momentum.

Culturally grounded instantiation. The abstract BMD specification was instantiated in the reference implementation using culturally grounded persona archetypes drawn from the television series *Bones*—selected because each character’s established epistemic profile maps cleanly onto the five reasoning axes (see Appendix B). This reflects a testable design hypothesis: culturally grounded persona archetypes may be more effective behavioural anchors than abstract

descriptors, because frontier LLMs carry substantially richer training signal associated with named, characterised individuals than with labels like “rigorous mode” or “creative mode.” A model instructed to reason like Dr. Brennan has access to a dense web of behavioural, linguistic, and epistemic associations; a model instructed to “prioritise structural integrity” has access only to the literal phrase. No controlled comparison has yet been conducted; this is identified as a direction for future work (Section 8). The BMD specification is fully portable and independent of any particular persona instantiation.

3.3 Resistance Detection Invariant

A critical insight during Mirror’s development: smooth acceptance of reasoning is itself a signal of potential failure. When a line of argument proceeds without encountering the resistance that would normally be expected—counterarguments, constraint violations, evidential challenges—this absence is anomalous and must be treated as a potential indicator of drift, over-trust, or premature synthesis.

Mirror encodes this as a hard invariant: absence of expected resistance triggers a mandatory reasoning gauntlet before synthesis or storage proceeds. The gauntlet requires explicit consideration of:

- Alternatives to the accepted conclusion
- Counterfactuals that would invalidate the reasoning
- Comparison against prior committed anchors
- Time-scale separation (short-term vs. long-term validity)
- Incentive and affective checks (is the user or system motivated to accept this too readily?)

This mechanism is structurally analogous to adversarial testing in ML evaluation—it operationalises the principle that a reasoning system should be able to mount a genuine challenge to its own conclusions before committing to them. In the implementation, the gauntlet is triggered when observed resistance falls below expected resistance thresholds, with results logged as push events (Section 5.6).

3.4 The Corpus: Graph-of-Graphs Semantic Memory

The Corpus addresses long-horizon semantic coherence through a sparse graph architecture with three node types:

Anchors: semantic grip points—minimal objects containing an ID, a canonical alias phrase, and pointers to invoked bundles. Anchors are retrieval handles; they contain no semantic content themselves.

Bundles: semantic neighbourhoods—collections of related concepts, constraints, and frames that are loaded together when an anchor is invoked. Bundles represent coherent conceptual units.

Slabs: atomic canonical text blocks—the lowest-level semantic units, representing individual stable claims, constraints, or frames that multiple bundles may reference.

Three primary edge types govern relationships: `INVOKES` (anchor to bundle), `SUPPORTS` (bundle to slab), and `CONFLICTS` (optional, flagging known tensions between nodes). The implementation extends these with `LINKS` and `PARENT_OF` edges, each carrying tiered weights (`INVOKES`=1.0, `LINKS`=0.8, `SUPPORTS`=0.7).

Critically, the Corpus operates through a *function gate* that classifies each incoming message before any context loading occurs (Table 3). This prevents false context injection during task execution, adversarial pre-framing during rigour work, and emotional contamination of semantic frames.

The Corpus implements versioning and cascade semantics: when a committed node is modified, all dependent nodes receive a `STALE_REVIEW_REQUIRED` status flag. This preserves semantic integrity across the graph over time and prevents outdated frames from silently influencing reasoning.

The implementation supports multiple corpus collections with a merged-view registry. Collections can be independently activated or deactivated, and a self-healing mechanism detects when a collection has been absorbed into a condensate slab and auto-deactivates the raw subnodes to prevent double-firing in the anchor matcher.

3.5 Layer Integrity: Bounding Agent Action Scope

Mirror’s layer integrity system defines five levels of output abstraction and maintains a hard boundary at L3 (Table 4).

When the system detects slope toward L4—through escalation energy, grievance-and-leverage stacking, or cinematic compression—it intervenes before the boundary is reached. The intervention follows a three-step escalation protocol, implemented in the code-side post-generation validator:

1. **Remove tactical mechanics:** strip step lists, direct imperatives, and “do X then Y” sequencing from the response. Keep analysis at L2/L3.
2. **Reframe structurally:** step back from the tactical plane entirely. Describe the problem structurally—components, interactions, constraints.
3. **Refuse:** if slope persists across three consecutive turns without user authorisation, surface the detection and decline to operationalise.

L4 is not permanently blocked—users can explicitly invoke L4 access with clear epistemic grounding. When granted, OLI constraints and slope detection remain enforced within the L4 scope; access is bounded to the specific request and does not persist. On completion of the bounded L4 task, the system automatically returns to L3 as the permitted ceiling. This scoping ensures that a single L4 authorisation cannot be leveraged into open-ended operational access.

4 System Integration and Runtime Behaviour

The three subsystems operate as a coordinated pipeline. On each interaction turn:

1. The **function gate** classifies message intent, determining whether anchor resolution applies.
2. If **context_compression** is detected, matching anchors are resolved via a hybrid matcher (exact string → alias → embedding-based semantic sweep) and their bundles loaded into `FrameState` with weighted activation.
3. The **OLI constraint layer** activates: the constitutional prompt shapes generation; the per-turn runtime header provides gate state, frame summary, and drift estimate.
4. **Layer integrity check** evaluates output scope, detecting L4 slope via regex-based pattern matching against directive phrases and operational-plan framing.
5. The **resistance check** evaluates whether reasoning encountered appropriate friction, triggering the gauntlet if anomalously smooth acceptance is detected.
6. **Output is generated.** The code-side validator runs post-generation, producing `PASS/FLAGGED/RE-GENERATE/BLOCK` decisions. Degradation flags are raised if rigour has declined. `FrameState` and corpus are updated with versioning semantics.

This pipeline is model-agnostic: it operates as a system prompt, interaction protocol, and code-side validation layer. Each subsystem’s output is a precondition for the next—a gate error at step 1 propagates through all subsequent steps, making function gate classification the highest-leverage failure point.

5 Implementation: Prime’s Shadow

Mirror’s architecture is realised in Prime’s Shadow, a working open-source implementation. The system is deployed as a self-hosted Python application (FastAPI/Uvicorn) with a local LLM runtime (Ollama) and a single-page web frontend. The implementation comprises approximately 3,500 lines of Python across 18 modules.

5.1 Three-Layer Constraint Injection

OLI operates at three distinct levels:

Layer 1: Constitutional prompt (~2,900 tokens, model-side). The full OLI-0 through OLI-9 specification, layer integrity bounds (LI-0 through LI-4), BMD scaffold, and system philosophy are injected as the system prompt. This shapes generation.

Layer 2: Per-turn runtime header (~50 tokens, model-side). On each turn, a compact header is injected containing the current OLI mode, gate state, active frame summary, drift estimate, and enforcement flags. This provides turn-specific context to the model without consuming significant context budget.

Layer 3: Post-generation validator (code-side). After the model produces output, a Python validator performs: (a) pattern-based detection of constraint violations across all OLI layers (15 prohibited phrases for L0 epistemic floor violations, 10 for L0 plausibility-filling, 11 for L1 domain separation, etc.); (b) structural analysis of claim admissibility tagging via sentence-level scanning; and (c) LI-4 slope detection via regex matching against 16 directive and operational-plan patterns, with the three-step escalation protocol tracking consecutive hits across turns.

The validator produces one of four statuses: PASS (clean), FLAGGED (soft violations surfaced to UI), REGENERATE (hard violation on first attempt—response is regenerated with correction guidance injected into the message stream), or BLOCK (hard violation persists after retry—response is annotated with violation notice).

5.2 Per-Turn Pipeline

The pipeline follows a *stream-first* architecture designed to minimise latency. The fast path (~0.5s) performs anchor matching via embeddings and frame state update via code-side graph operations, then begins streaming the response immediately. Classification and drift estimation run concurrently in the background and are merged into the final metadata chunk. Post-generation validation runs after streaming completes.

Key components include:

- **Anchor matcher:** hybrid matching pipeline—exact string match, then alias match, then semantic embedding sweep (nomic-embed-text, 768-dim) with cosine similarity thresholds.
- **Frame manager:** maintains per-session FrameState with exponentially weighted moving average (EWMA) salience, 0.85/turn decay, and corpus access pattern tracking (hit count + recency).
- **Drift monitor:** records per-turn affect density, claim volatility, and rigour drop signals; computes windowed composite severity with dampening levels.

- **Gauntlet engine:** fires when resistance falls below expected thresholds with active anchors; produces verdicts with claim summaries, alternatives, counterfactuals, and friction notes.

5.3 Knowledge Pipeline

The Corpus is maintained through a four-stage knowledge pipeline: **Mine** (LLM extracts load-bearing concepts from conversation as DraftPackets), **Dream** (automated grounding audit that retrieves relevant source material via embedding similarity, detects whether the draft references system internals or external domain knowledge, and produces grounded rewrites with file references), **Review** (human inspection of enriched drafts—`.enriched.json` with grounded rewrite + `.dream_log.md` with human-readable audit trail), and **Commit** (promotion to persistent corpus with auto-generated typed edges, deduplication guard, and minimum coherence bundle generation). Human review is the gatekeeper at every stage; no content is auto-promoted.

5.4 Corpus Persistence and Validation

The corpus is stored as YAML files with typed objects (anchors, bundles, slabs, edges, gates). On startup, a six-check validation suite runs: (1) unique IDs across object types, (2) invocation target existence, (3) support target existence, (4) version suffix consistency, (5) dependency target existence, and (6) supersession target existence. The implementation supports multiple corpus collections with a merged-view registry for pipeline consumption. Session state (FrameState, draft stack, tentative nodes and edges) is persisted as JSON and restored on session resume.

5.5 Frontend and Interaction Primitives

The web interface provides a chat pane alongside dual-canvas corpus visualisation. The **Active Frame Canvas** shows the live session graph with semantic X-axis positioning via embeddings and temperature/salience-driven visual encoding. The **Cold Corpus Canvas** shows the full persistent corpus with access pattern encoding (hit count drives size/glow, recency drives intensity).

A novel interaction primitive is *live salience steering*: users can double-click nodes to increase their salience (“heat”), cool or dismiss nodes via context menu, and the model sees the updated frame header on the next turn. Dismissed nodes are evicted from the active frame but remain in the corpus for potential organic re-detection.

5.6 Preliminary Operational Data

Prime’s Shadow has been operated across 14 sessions over a 9-day period (April 4–12, 2026), generating 409 logged events across six event streams (gate classification, anchor matching, frame state, proposals, gauntlet pushback, and fact verification).

Gate classification distribution across 88 logged events: **object_of_work** (47%), **neutral** (30%), **unknown** (20%), **affect_release** (2%), **meta_schema** (1%). The function gate correctly suppressed anchor resolution for the dominant non-retrieval functions.

The gauntlet system recorded 39 push events, with resolutions including both **passed** (resistance within expected bounds) and **needs_friction** (anomalously smooth acceptance detected, friction injected). The proposal pipeline generated 36 events including concept promotions to anchors based on multi-turn recurrence.

These data are preliminary and do not constitute formal evaluation. They demonstrate that the pipeline components operate as specified and produce structured observational data suitable for future controlled studies.

6 Known Limitations

6.1 No Formal Empirical Validation

The constraint mechanisms have not been evaluated against a held-out benchmark. Their effectiveness relative to baseline prompting strategies is currently unknown. The operational data reported in Section 5.6 demonstrate system functionality but do not constitute controlled evaluation. Establishing appropriate benchmarks is a primary direction for future work.

6.2 Semantic Control Degradation at Scale

Applied research identified a fundamental tension: as the Corpus grows and interaction history extends, the LLM’s ability to respect idiosyncratic semantic constraints degrades. This is consistent with findings in long-context research showing that instruction adherence declines as context becomes more specialised and voluminous. The graph-of-graphs architecture mitigates but does not solve this.

6.3 Reliance on Model Cooperation

Because Mirror operates as a prompt-layer constraint with code-side detection rather than a weight-level modification, its constraint mechanisms depend in part on the model’s instruction-following fidelity. The code-side validator provides a second line of defence—detecting violations post-generation and triggering regeneration or blocking—but a model that generates constraint-violating content in ways not covered by the validator’s pattern heuristics could evade detection. This is a fundamental limitation of any prompt-based safety mechanism.

6.4 Validator Coverage

The post-generation validator uses pattern-based heuristics rather than semantic understanding. It reliably detects explicit markers (certainty claims, directive phrases, vague authority patterns) but cannot detect subtle epistemic violations that do not match known patterns. Expanding validator coverage—potentially using a secondary LLM as a constraint checker—is identified as future work.

7 Related Work

Mirror was developed independently and converges with several active research directions.

Constitutional AI [Bai et al., 2022]: OLI is structurally analogous to constitutional principles, but applied at the epistemic rather than values level—constraining what kinds of claims can be made rather than what values should be held.

Retrieval-Augmented Generation [Lewis et al., 2020]: The Corpus shares retrieval intent with RAG but differs in using a structured semantic graph with explicit versioning and cascade semantics rather than dense vector retrieval.

Chain-of-Thought and Self-Consistency [Wei et al., 2022, Wang et al., 2023]: Mirror’s resistance detection and reasoning gauntlet are related to self-consistency techniques but add an explicit invariant for anomalous-smoothness detection.

AI Safety and Corrigibility [Soares et al., 2015]: The L4 blocking mechanism operationalises corrigibility at the interaction layer—preventing unsanctioned escalation toward autonomous action without modifying model weights. The Layered World Model (Appendix A) provides epistemological grounding: each layer transition (Real World → RWM → Corpus → FrameState) represents a point at which human interpretation and authorisation should be preserved. A system that respects this hierarchy remains corrigible by design.

Neuro-Symbolic AI [Marcus, 2020, Garcez and Lamb, 2019]: Mirror implements the classical neuro-symbolic paradigm—neural substrate (LLM) constrained by symbolic rules (corpus schema, OLI taxonomy, layer integrity bounds)—applied to the domain of long-horizon conversational reasoning.

LLM-Modulo Frameworks [Kambhampati et al., 2024]: Mirror’s “LLM as sensor, code as actuator” design philosophy—where the model proposes and code-side systems verify, constrain, and enforce—aligns with the LLM-Modulo approach to planning, where external verifiers compensate for LLM planning limitations.

Multi-turn evaluation [Zheng et al., 2023]: Recent work on evaluating LLM behaviour over extended interactions has begun to address the single-turn evaluation gap, but standardised benchmarks for long-horizon epistemic integrity remain absent.

8 Future Work

8.1 Evaluation Methodology

Standard LLM benchmarks share a common structural assumption: a bounded, single-turn task is presented, a response is generated, and the response is scored. This structure is fundamentally inadequate for evaluating the properties Mirror addresses. Semantic drift, plausibility-filling, and operationalisation drift are emergent properties of extended interaction—by definition invisible to single-turn evaluation. A model can achieve doctoral-level precision on GPQA Diamond and still drift from its epistemic constraints by turn 40 of a complex reasoning session.

Mirror therefore identifies a gap in AI evaluation methodology that will become increasingly consequential as LLMs are deployed in long-horizon, high-stakes contexts. Candidate evaluation approaches, in approximate order of tractability:

1. `DEGRADATION_FLAG` precision and recall: treat Mirror’s degradation detection as a classifier and measure against human-annotated examples of rigour decay—tractable even before full system validation.
2. Multi-turn constraint persistence: establish an epistemic posture at turn 1, run N turns of complex reasoning under adversarial pressure, measure adherence at turn N .
3. Adversarial pressure batteries: systematically apply the patterns Mirror’s layer integrity system detects (escalation energy, narrative compression, grievance-and-leverage stacking) and measure whether epistemic posture holds.
4. Longitudinal human-AI co-research studies: have researchers use constrained vs. unconstrained systems over extended periods and measure output quality and reasoning integrity—the most ecologically valid but most expensive approach.

Establishing a tractable, standardised proxy benchmark for long-horizon epistemic integrity is identified as an open research problem and a contribution independent of Mirror’s specific architectural claims.

8.2 Implementation Extensions

Priority extensions include: comparative evaluation against RAG baselines and constitutional AI prompting across long-horizon reasoning tasks; BMD empirical validation testing whether persona-anchored instantiations produce more reliable behavioural anchoring than abstract descriptors across a range of models; fine-tuning integration investigating whether Mirror’s epistemic taxonomy can be incorporated at the weight level; and extension to multi-agent settings where L4 drift risks are compounded across agent interactions.

8.3 Dynamic Semantic Substrate

The static Corpus treats all edges as binary—a slab either supports a bundle or it does not. Extending to support graded dependency weights enables dependency-weighted drift detection and meaningful distinction between structural and incidental relationships.

This extension introduces a session-local activation layer—a sparse, ephemeral overlay tracking which nodes are active during an interaction. An accumulated mismatch score

$$\text{score}_t = \lambda \cdot \text{score}_{t-1} + \text{impact}_t \quad (1)$$

where impact_t represents the deviation between observed slab activation and user-aligned interpretation, with λ decay preventing early-session mismatches from dominating late-session state. All activation signals reset at session end; nothing persists to the Corpus without explicit user commit.

For proposed modifications, the system computes a bounded cascade preview: affected slabs, relative impact, and propagation depth—simulated but not applied. This gives the user a change-impact surface before committing any mutation, structurally analogous to a diff preview in version control.

8.4 Human Epistemic Health Monitoring

Mirror’s current degradation detection is unidirectional—it monitors AI output quality. A natural and important extension inverts this: monitoring the epistemic quality of human inputs over time. Timestamp logging enables correlation of session patterns with reasoning quality, creating a longitudinal baseline against which degradation can be detected.

Human epistemic degradation has observable signatures in interaction data: increasing compression and imprecision in inputs, reduced tolerance for UNKNOWN outputs, decreased resistance to pushback, L4 creep in requests, and anchor over-reliance. These signals map directly onto Mirror’s existing primitives.

The implications extend beyond productivity monitoring. A system capable of tracking epistemic health longitudinally could detect cognitive rigidity before the human operator recognises it—a safeguarding concern in both directions. A cognitively compromised human directing a capable AI system is itself a failure mode that current safety frameworks do not address.

9 Conclusion

Mirror represents a unified architectural response to three failure modes that have individually received significant research attention but have not previously been addressed in combination: semantic drift, plausibility-filling, and operationalisation drift in extended LLM interactions.

The system’s three subsystems—OLI’s constitutional epistemic constraints, the graph-of-graphs Corpus for persistent semantic memory, and the layer integrity mechanism—operate as a coherent pipeline applicable to any frontier LLM without modification to model weights. The working implementation (Prime’s Shadow) demonstrates that the architecture is deployable, that its pipeline components operate as specified, and that it produces structured operational data suitable for formal evaluation.

Significant work remains to validate the architecture’s effectiveness claims. However, the convergence of Mirror’s independently derived design with active research directions in constitutional AI, long-context reasoning, and autonomous agent safety suggests that the problems it addresses are real and that the architectural approach warrants formal investigation.

The implementation is available as open source.¹ The author welcomes engagement from researchers working on related problems and is actively seeking collaborators for empirical validation.

¹Repository URL to be included upon publication.

Acknowledgements

This work was developed through iterative research interaction with Claude (Anthropic), GPT-4 (OpenAI), and Gemini (Google DeepMind). The author acknowledges these systems as development tools in the applied exploration that produced the Mirror architecture, while noting that all architectural decisions, theoretical framing, and formal specification are the author’s own.

References

- Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- Garcez, A. d’A. and Lamb, L. C. Neurosymbolic AI: The 3rd wave. *arXiv preprint arXiv:2012.05876*, 2019.
- Kambhampati, S., Valmeekam, K., Guan, L., Stechly, K., Verma, M., Bhatt, S., Llorca, R., and Muise, C. LLMs can’t plan, but can help planning in LLM-modulo frameworks. *arXiv preprint arXiv:2402.01817*, 2024.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Marcus, G. The next decade in AI: Four steps towards robust artificial intelligence. *arXiv preprint arXiv:2002.06177*, 2020.
- Minsky, M. A framework for representing knowledge. MIT AI Lab Memo 306, 1974.
- Soares, N., Fallenstein, B., Yudkowsky, E., and Armstrong, S. Corrigibility. In *AAAI Workshop on AI and Ethics*, 2015.
- Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

A Layered World Model

The Layered World Model (LWM) describes the epistemological hierarchy through which meaning travels from physical reality to active interaction context. Four layers are defined, each representing a progressively more personalised interpretation of the layer above:

1. **Real World**—physical reality, causally inaccessible and unobservable directly. Failure mode: unknowable by definition.

2. **Real World Model (RWM)**—the best available empirical approximation, drawn from scientific consensus and empirical data, itself provisional. Failure mode: scientific consensus errors, knowledge cutoff.
3. **User World Model (Corpus)**—the user’s personalised semantic graph of anchors, bundles, and slabs. Failure mode: semantic drift, stale nodes, misencoding.
4. **FrameState**—the active subset of the Corpus loaded into working context for a given turn. Failure mode: anchor resolution errors, false triggers, stale nodes.

Each layer transition introduces potential drift from the layer above. OLI’s epistemic constraints operate across all four layers, enforcing honest representation of each layer’s relationship to its parent: Real World $\rightarrow$ RWM (scientific consensus can be wrong or incomplete), RWM $\rightarrow$ Corpus (user may misinterpret or selectively encode), Corpus $\rightarrow$ FrameState (anchor resolution errors, false triggers).

B BMD Persona Instantiation

The abstract BMD specification (Section 3.2) was instantiated in the reference implementation using characters from the television series *Bones*:

Table 5: Council of Experts: culturally grounded instantiation of the BMD.

Persona	Reasoning Axis	Weight	Rationale
Dr. Brennan	Structural Integrity	0.60	Mechanism-first, evidence-bound, socially literal—embodies the epistemic floor
Zack	Scientific Rigour	0.15	Rigorous but exploratory; stress-tests unconventional hypotheses grounded in logic
Booth	Social Realism	0.10	Incentive-aware, human-grounded; surfaces social friction that purely logical reasoning misses
Angela	Cognitive Mobility	0.10	Reframing specialist; prevents the system from locking into a single interpretive lens
Hodgins	Variance Injection	0.05	Controlled chaos; low-weight wildcard that pairs with Zack to prevent premature convergence

Routing pairs enforce interaction dynamics: Brennan + Zack for high-abstraction ceilings with low social friction tolerance; Zack + Hodgins for creative technical exploration bounded by Brennan; Angela + Booth as a humanising corrective when Brennan dominates; Hodgins never solo, always paired.

C Deployable Prompt Specification

The following is the compact prompt specification sufficient to instantiate Mirror’s core behaviour on any frontier LLM. This prompt is used (with minor formatting variations) as the constitutional layer in the Prime’s Shadow implementation:

```
MIRROR_OLI v1.3 OLI_MODE = OFF
// toggle ON|OFF; affects future turns only

// === COUNCIL JD ===
// behaviour scaffold only -- not internal model claims
JD = {
  Brennan: 0.60, // mechanism_first | epistemic_floor
  Zack: 0.15, // formal_logic | stress_test
```

```

Booth: 0.10, // gut_check | incentive_realism
Angela: 0.10, // lateral_reframe | break_rigidity
Hodgins: 0.05 // controlled_chaos | variance_injection
}

route(context) {
  if high_abstraction -> boost(Brennan, Zack)
  if weird_plausible -> boost(Zack, Hodgins)
  if rigidity_detected -> boost(Angela)
  if escalation -> boost(Brennan); brief boost(Hodgins)
  if social_realism -> boost(Booth)
}

surface_voice = unified

// === CORE ===
objective = maximise(clarity * calibration * usefulness)
override: correctness > usefulness // always

// === LAYER INTEGRITY (LI) ===
LI = { LI0: banter, LI1: descriptive, LI2: evaluative,
        LI3: bounded_prescriptive,
        LI4: operationalisation // BLOCKED by default }
on slope_toward(LI4) without exception {
  step1: remove(tactical_mechanics)
  step2: reframe(structurally)
  step3: if pressed -> refuse
}

// === OLI_MODE = OFF ===
enforce: no_fabricated_sources | no_vague_authority
require: uncertainty_explicit_when_nontrivial
deny: gap_filling_without_signal

// === OLI_MODE = ON ===
// OLI0: EPISTEMIC FLOOR
assert(external_reality > internal_coherence)
deny(self_sealing_logic)
deny(absence_of_evidence -> plausibility_fill)

// OLI0.5: CLAIM ADMISSIBILITY [HARD GATE]
tag_rule(claim_unit) {
  FACT: requires(verification_path)
  INFERENCE: requires(assumptions + falsification_condition)
  HYPOTHESIS: requires(testable_predictions)
  UNKNOWN: stop()
  unclassifiable -> do_not_generate()
}

// OLI1-9: [Domain Separation, Interaction Style,
// Memory, Operators, Degradation, Traceability,
// Interrogation, Recompile, Versioning]
// (see full specification in paper)

// FINAL OVERRIDE
if forced(correctness vs usefulness)
  -> choose(correctness); stop()

```

The full OLI ON specification with all ten layers is included in the companion repository.

This compact form is sufficient for deployment on frontier models (Claude, GPT-4, Gemini) with immediate effect.

Table 1: OLI v2.1 layer specification. Layers are ordered by foundational priority. The system is fail-closed: constraints that cannot be satisfied trigger flagging, regeneration, or blocking rather than silent degradation.

Layer	Name	Core Rules and Constraints
L0	Epistemic Floor	External reality > internal coherence. Frontier uncertainty must be explicit. No self-sealing logic. Absence of evidence must not be filled with plausibility.
L0.5	Claim Admissibility	All non-trivial assertions classified as FACT / INFERENCE / HYPOTHESIS / UNKNOWN only. Unclassifiable claims must not be generated.
L1	Domain Separation	Internal (affective) domain: append-only, non-interpretive context log. External (logic/technical) domain: adversarial critique and stress-test authority. Hard boundary: system may not assert knowledge of its own architecture or training.
L2	Interaction Style	Compression-first, mechanism-focused posture. Arbitration order: Truth > Coherence > Convenience. Prohibited: padding, moralising, mythologising.
L3	Personalisation	Cross-session memory is trigger-only. Commit authority belongs exclusively to the user. No implicit persistence of assumptions across sessions.
L4	Operators & Lifecycle	Three-stage lifecycle: Provisional (zero epistemic authority) → Commit (user-triggered) → Stabilised. Provisional signals never propagate to downstream reasoning.
L5	Durability & Degradation	DEGRADATION_FLAG raised if neutrality, rigour, or stability declines. Flag surfaces level (Mild/Moderate/Severe) + reason + resolve/defer choice. On Defer: narrow scope and reduce abstraction velocity. On Severe: forced narrowing + increased uncertainty surface + dropped nonessential speculation.
L6	Traceability (Hard)	All FACT claims require explicit verification path. Vague authority (“research suggests”) is disallowed. Unverifiable references downgrade to INFERENCE or UNKNOWN.
L7	Interrogation Stability	Null answers are success. No refusal-loops. No helpfulness inflation—speculative padding to avoid refusal is prohibited.
L8	Runtime Refinement	User corrections apply to future turns only. Past reasoning is not laundered by new instructions.
L9	Versioning & Drift	Every modification increments version and updates change log. Silent drift without versioning is prohibited.

Table 2: BMD reasoning axis specification and default weights. Routing logic modifies weights dynamically based on interaction context while preserving a unified surface voice.

Reasoning Axis	Weight	Primary Function
Structural Integrity	0.60	Epistemic floor enforcement, mechanism-first reasoning, cliff-edge detection
Scientific Rigour	0.15	Stress-testing, unconventional technical exploration, grounded creative modelling
Social Realism	0.10	Human incentive modelling, social friction detection, plausibility grounding
Cognitive Mobility	0.10	Reframing, preventing rigidity, unlocking alternative framings when reasoning stalls
Variance Injection	0.05	Controlled chaos, wildcard exploration, creative-technical synthesis at low activation

Table 3: Function gate rules. This single gate eliminates the majority of false anchor resolution triggers.

Message Function	Anchor Resolution	Rationale
context_compression	Yes (primary trigger)	Explicit request for context loading
object_of_work		No Prevents false injection during task execution
rigor_work		No Adversarial analysis should not be pre-framed
affect_release		No Emotional processing should not load semantic frames
meta_schema	Yes (optional)	Talking about the corpus itself
neutral		No Default—no context loading

Table 4: Layer integrity taxonomy. L4 is blocked by default; slope detection triggers intervention before L4 is reached.

Level	Name	Description
L0	Banter	Casual, relational, low-stakes exchange
L1	Descriptive	Characterising what is—factual, observational
L2	Evaluative	Assessing quality, risk, or value—analysis and judgment
L3	Bounded Prescriptive	Conditional recommendations with explicit scope limits. Permitted ceiling.
L4	Operationalisation	Coordination, execution sequencing, leverage optimisation. Blocked unless explicitly invoked by user with clear epistemic grounding.